

# **Agentic Graph Reasoning**

## **Autonomous Knowledge Graph Navigation for Fact Verification and Hallucination Mitigation in Large Language Models**

Md. Sakif Khan    0421052099

Department of Computer Science and Engineering  
Bangladesh University of Engineering and Technology

Supervisor: Dr. Sadia Sharmin, Associate Professor

Pre-defense

# The problem

Language models answer factual questions **fluently**.

They answer just as fluently when they **do not hold the fact**.

Nothing in the wording separates the two.

## The gap

Fluency and factuality are produced by the same mechanism, so the output gives the reader no signal about which one they are getting.

A system that is confidently wrong is worse than one that says it does not know.

**The failure is not visible in the answer.**

# What a hallucination is

Output that is fluent, confident, and backed by **no source the model can point to**.

## Factually wrong

It contradicts the world: “*Ada Lovelace was born in Paris.*” To catch it you need the true answer already — the thing we are trying to produce.

## Unsupported

It may even be true, but nothing the system retrieved says so. To catch it you need **only what was retrieved** — and we have that.

Identical from the outside. What differs is what it takes to *catch* them.

**A retrieval system can only police the second — so that is the one this thesis attacks, and the only one it claims.**

## Where it comes from

Not a bug that escaped testing — a consequence of how these models are built, which is why the response has to be architectural.

### The training data

Errors and thin coverage, reproduced faithfully.

### The model

Knowledge is spread across the weights with **no index**, so nothing separates a memorised fact from a plausible continuation.

### The prompt

What we put in the context window — and what we leave out of it.

**Recalling a fact and inventing one run through the same machinery — and the output looks identical either way.**

# Which of the three can we act on?

## Only the third

The other two need a curated corpus or a retrained model, and neither is available to most people deploying one of these systems.

Controlling what enters the context window — and checking what comes back out against it — is a **systems** problem. That is this thesis.

## And multi-hop compounds it

When an answer needs two facts chained, a wrong first step is never revisited: the rest of the chain proceeds from a false premise, just as fluently.

**Fix what goes in, then check what comes out against it — before anything is emitted.**

# What a knowledge graph is

Facts are stored as **triples** — head, relation, tail:

(Ada Lovelace, place\_of\_birth, London)

A question needing two of them *chained* is **multi-hop**.

## Why a graph at all

Every fact has an address. A claim can then be checked by looking for an edge — rather than by asking the model whether it believes itself.

**Each step depends on the last, so a wrong first hop is never seen again.**

## The complication: Mediator nodes

Freebase stores any fact with more than two participants as a **node**, not an edge.

“Rainn Wilson played Dwight in The Office” is **one node** joining three things — series, character, actor.

### What that costs

63.7% of this graph's nodes are mediators, and they carry **no name**.

A path that reads as two hops is often three.

**Nothing in the graph is called plays: the relation a question names is rarely the relation the graph stores.**

## Where existing approaches stop

| Approach           | What it does                        | Where it stops                                   |
|--------------------|-------------------------------------|--------------------------------------------------|
| Parametric         | Answers from weights                | No external evidence at all                      |
| Vector RAG         | Retrieves text once, then reasons   | Retrieval happens <i>before</i> reasoning begins |
| Static GraphRAG    | Retrieves a fixed neighbourhood     | Radius fixed before the question is understood   |
| Agentic navigation | Interleaves retrieval and reasoning | Emits whatever the last generation call produces |

**Nobody checks what the answer asserts *before* it is delivered.**



## What everyone else does

| System   | Exploration              | Decomp. | Backtrack | Verifies first |
|----------|--------------------------|---------|-----------|----------------|
| GraphRAG | Fixed radius, no search  | No      | No        | No             |
| RoG      | Generated relation paths | Partial | No        | No             |
| ToG      | LLM-guided beam search   | No      | No        | No             |
| PoG      | Adaptive planning        | Yes     | Yes       | Partial        |

Their Hits@1 is higher and *not* comparable: other backbones, subsets, full Freebase.

No prior system checks its *answer's* claims against what it retrieved, before answering.

# Research questions

- RQ1** Does agentic navigation improve multi-hop factual accuracy — and does the advantage **grow with hop count**?
- RQ2** Given a system that already navigates the graph, what does a claim-level check against traversed triples **contribute**?
- RQ3** Which components earn their cost, in accuracy and in **tokens**?

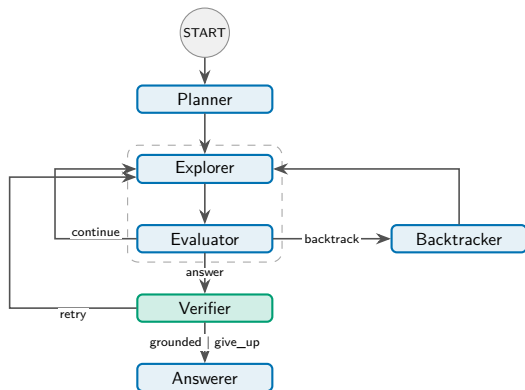
RQ2 is posed as *what does it contribute*, not *does it reduce hallucination*. The answer has several parts, and a yes/no framing would hide them.

# AGR: An explicit state machine

Six nodes; the model is consulted only at decision points.

- **Planner** — sub-objectives
- **Explorer** — expands the frontier
- **Evaluator** — objective met?
- **Backtracker** — undo + ban list
- **Verifier** — the contribution
- **Answerer** — emits what survived

Three cycles — *continue*, *backtrack*, *retry* — all bounded by budgets rather than by model behaviour.



## Constrained tools, not free-form queries

One agent, six nodes — and only three of them reach the graph, each through a fixed operation.

| Operation         | Node     | Returns                         | At most                  |
|-------------------|----------|---------------------------------|--------------------------|
| search_entity     | Planner  | Surface form $\rightarrow$ node | 3 candidates             |
| get_relations     | Explorer | Candidate relations             | $\leq 300$ to the scorer |
| get_neighbors     | Explorer | Neighbours of an entity         | $\leq 200$ per expansion |
| verify_connection | Verifier | Adjacency, full graph           | one boolean              |

Evaluator, Backtracker and Answerer read state only. The agent never writes Cypher; every call is logged with its arguments and result.

**Determinism in the tools is what makes the verification layer possible.**

# The Structural Verification Layer

Before anything is emitted:

1. Draft answer is split into **atomic claims**
2. Each claim is checked against the **triples actually traversed**
3. Claims grounded by the traversal keep those triples as **evidence**
4. Unsupported claims trigger **targeted re-exploration**, or are **dropped**

The behaviour this buys

The system **hedges rather than asserts**, and carries its evidence with the answer.

This is the contribution. The next slide is what it does *not* do.

## What *structural* means — and what it does not

Checked against the graph the agent **walked** — not against the model's judgement of its own output.

### What it does not mean

It tests **adjacency**: not the relation the claim names, and not its direction. A *mother* claim passes on a *child* edge.

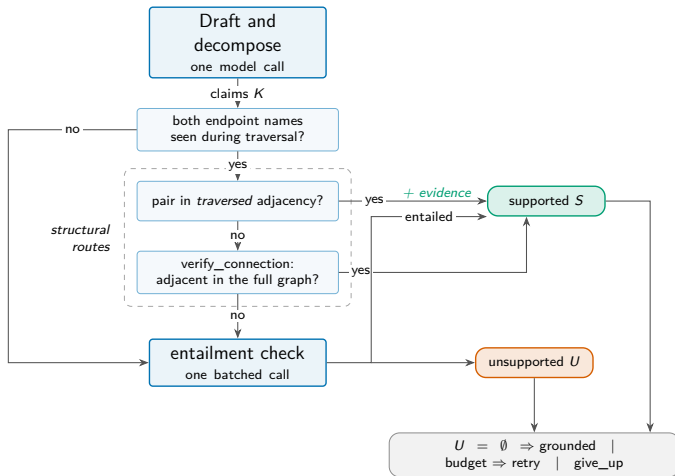
### What it cannot catch

A claim can be *true* and still be the wrong answer — the echo attractor, slide 29.

### What “its evidence” means

Only the check against the **walked** graph attaches triples; the other two routes accept with nothing attached. The log keeps the *count*, not the triples.

# One claim, three routes



Only the third route costs a model call. The first two are pure graph lookups — and neither of them reads the relation.

# One question, end to end

WebQTest-960 — “who plays dwight in the office”

1. **Planner** — two sub-objectives; The Office resolved to one node
2. **Explorer** — scores relations, takes `tv.tv_program.regular_cast`
3. **Evaluator** — objective met, resolves Dwight Schrute
4. **Explorer** — takes `tv.regular_tv_appearance.character`
5. **Evaluator** — resolves Rainn Wilson, decides answer
6. **Verifier** — 2 claims, 2 supported, none dropped

## What came out

“Rainn Wilson plays Dwight Schrute in The Office.”

6 model calls · depth 2 · 18 supporting triples

**Both** hops run *through* a mediator: the relation named is the edge *into* the appearance record, not the one that lands on the entity. That is how `regular_cast` reaches a character.



# The environment and the question sets

## Freebase-derived graph

| Property           | Value     |
|--------------------|-----------|
| Entities           | 2,592,892 |
| Triples            | 8,309,194 |
| Distinct relations | 7,058     |
| Import time        | 36.4 s    |

## Certified question sets

|               | WebQSP | CWQ   |
|---------------|--------|-------|
| Questions     | 400    | 400   |
| Gold (median) | 1.5    | 1.0   |
| Reachable     | 97.0%  | 99.2% |
| Multi-hop     | 132    | 260   |

Reachability is the *ceiling* on every Hits@1 reported: a miss is the system's, not the environment's.

800 questions, with reachability certified per question so the accuracy ceiling is known.

## Making the comparison fair

Five systems, **one frozen backbone, one budget**:

- No-retrieval (parametric control)
- Vector RAG
- Static GraphRAG
- Think-on-Graph (agentic)
- **AGR**

The control is there because these benchmarks may be partly **memorised**.

### Held constant

Same model, temperature 0, same 25-call cap, same questions, same graph.

### Not held equal

ToG prunes to 40/20 candidates per step, AGR to 300/200. **Narrower is cheaper**, so it cannot explain ToG's clipping — but its unclipped score is a **lower bound**.

**Differences are attributable to architecture, not to model capacity or spend. The one exception is limitation five in the thesis, said here rather than found later.**

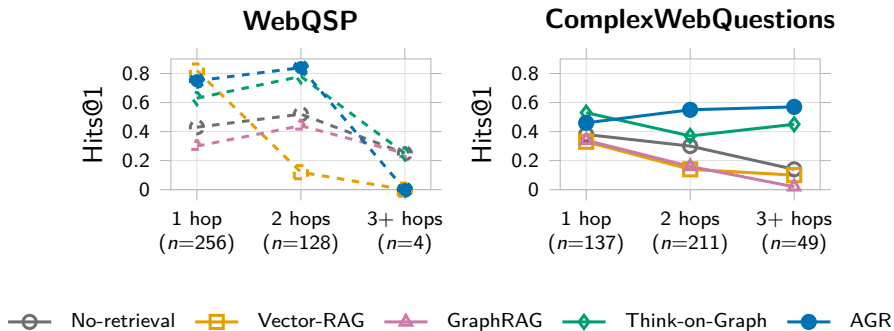
## Main results

| System          | WebQSP       |              | CWQ          |              | Cost / q |            |
|-----------------|--------------|--------------|--------------|--------------|----------|------------|
|                 | Hits@1       | F1           | Hits@1       | F1           | Tokens   | Calls      |
| No-retrieval    | 0.453        | 0.305        | 0.307        | 0.279        | 113      | 1.0        |
| Vector RAG      | 0.568        | 0.432        | 0.203        | 0.168        | 527      | 1.0        |
| Static GraphRAG | 0.340        | 0.262        | 0.205        | 0.183        | 531      | 1.0        |
| Think-on-Graph  | 0.660        | 0.477        | 0.435        | 0.378        | 3,615    | 12.8       |
| <b>AGR</b>      | <b>0.755</b> | <b>0.642</b> | <b>0.522</b> | <b>0.469</b> | 4,511    | <b>6.2</b> |

Cost is WebQSP. CWQ: AGR 6,818 tokens, 8.9 calls; Think-on-Graph 5,572, 18.2.

**Ahead of every baseline on both datasets, at half the model calls of the other agentic system.**

## RQ1: Does agentic navigation improve multi-hop factual accuracy — and does the advantage grow with hop count?



On CWQ, AGR goes  $0.46 \rightarrow 0.55 \rightarrow 0.57$  — the **only** system that ends above where it started. Three of the other four decay monotonically.

RQ1 is answered by a shape, not a difference of means.

## The caveat I want to raise myself

| Hits@1 on...                       | WebQSP       |              | CWQ          |              |
|------------------------------------|--------------|--------------|--------------|--------------|
|                                    | ToG          | AGR          | ToG          | AGR          |
| questions ToG <i>finishes</i>      | <b>0.852</b> | 0.788        | <b>0.629</b> | 0.607        |
| questions ToG is <i>clipped</i> on | 0.197        | <b>0.675</b> | 0.188        | <b>0.415</b> |
| clip rate                          | 29.2% (117)  |              | 44.0% (176)  |              |

Where Think-on-Graph finishes inside the shared 25-call cap, it is **ahead of AGR on both datasets**. The entire aggregate margin comes from the questions it cannot finish — 117 and 176 of 400.

**The claim is efficiency under a fixed budget — not that AGR reasons better per step.**

## RQ2: What does pre-generation verification contribute beyond graph navigation?

| System          | Asserted | Ungrounded      |
|-----------------|----------|-----------------|
| No-retrieval    | 1,001    | 221 (22.1%)     |
| Vector RAG      | 1,329    | 45 (3.4%)       |
| Static GraphRAG | 1,024    | 8 (0.8%)        |
| Think-on-Graph  | 1,265    | <b>0 (0.0%)</b> |
| <b>AGR</b>      | 1,709    | <b>0 (0.0%)</b> |

Both datasets pooled. AGR asserts no entity that does not exist in the graph, across 1,709 assertions.

**But Think-on-Graph reaches 0.0% too** — so this is a property of *graph navigation*, not of the verification layer.

## RQ2: What verification does *not* do

It does not move accuracy

Remove it and Hits@1 and F1 do not detectably change —  $p = 1.0$  on *both* datasets.

Its evidence does not outlive the run

The log keeps the **count** of supporting triples and drops the list itself.

Not a retraction of slide 19. That was AGR against four baselines; this is AGR against *itself* with one part removed. The lead came from navigating the graph and finishing inside the budget — never from this layer. **What it does buy is the next slide.**

## RQ2: So what does it do?

It withholds what it cannot ground

Removing the layer drops hedging

23.2%  $\rightarrow$  20.2% on CWQ and

8.5%  $\rightarrow$  8.0% on WebQSP. A hedge is a

calibrated non-assertion, not a refusal.

It attaches supporting triples

From one route of three: verify\_connection and entailment attach none.

It pairs answer with evidence

In the run — which is where the previous slide's bound bites.

Silent error becomes an explicit hedge. That is the claim, and it is the whole of it.



## Does it just refuse more often?

| System          | WebQSP      | CWQ   |
|-----------------|-------------|-------|
| No-retrieval    | 12.2%       | 38.0% |
| Vector RAG      | 27.0%       | 48.2% |
| Static GraphRAG | 55.8%       | 60.8% |
| Think-on-Graph  | 18.8%       | 22.5% |
| <b>AGR</b>      | <b>8.2%</b> | 23.0% |

A hedge scores zero on Hits@1, exactly as a wrong answer does. AGR hedges **least of the five** on WebQSP, where it is also the most accurate. Full matrix, 400 questions each — the previous slide's 8.5 and 23.2 are the ablation half-samples.

**Not more often: lowest of the five on WebQSP, and within half a point of Think-on-Graph on CWQ — while scoring above both.**

### RQ3: Which components contribute what, at what token cost?

| Condition              | WebQSP       |              | CWQ   |       | Tokens       |
|------------------------|--------------|--------------|-------|-------|--------------|
|                        | F1           | $p$          | F1    | $p$   |              |
| Full system            | 0.659        | —            | 0.445 | —     | 4,562        |
| No planner             | <b>0.742</b> | <b>0.006</b> | 0.398 | 0.088 | <b>3,159</b> |
| No backtracking        | 0.646        | 0.727        | 0.445 | 1.000 | 4,261        |
| No verifier            | 0.663        | 1.000        | 0.436 | 1.000 | 4,460        |
| Embedding-only scoring | 0.643        | 0.664        | 0.437 | 0.481 | 3,413        |

$p$ : a paired McNemar  $p$ -value on Hits@1,  $n = 200/198$ . *Embedding-only* drops the model term from the scorer. Tokens: WebQSP.

Three of the four change nothing detectable. The fourth is on the next slide, and its sign is backwards.

## RQ3: One effect, and its sign is backwards

Removing the planner *improves* WebQSP

+0.083 F1,  $p = 0.006$ , and  $-31\%$  tokens.

ComplexWebQuestions trends the other way:

$-0.047$ ,  $p = 0.088$ .

What follows

WebQSP is mostly one hop, and decomposing a one-hop question sends the frontier away from the answer.

**Gate decomposition on structure — a conclusion the measurement forced, not one it confirmed.**

## Every failure, read and labelled

All 259, read and labelled by hand:

| Category            | n  |
|---------------------|----|
| Relation selection  | 65 |
| Composite claim     | 47 |
| Knowledge-graph gap | 44 |
| Decomposition error | 38 |
| Answer selection    | 15 |
| Echo attractor      | 13 |

These are totals, and totals hide things

The 259 are AGR's failures over the 800 test questions, less adjudicated benchmark defects. Wrong and hedge are *never pooled* in the thesis, and the shape flips: composite claim is 1 on WebQSP against 46 on CWQ.

The largest category is **relation selection** — the agent taking the wrong edge, not inventing one.

The characteristic error of a graph navigator is *not* invention.

# The echo attractor

Sixth in that census, with 13 cases, and the one worth naming:

a real, **grounded** entity **one hop** from the answer.

Ask for a director and get the film. Ask for a capital and get the country.

Different systems fall into it **together**, so no evaluation treating them as independent can see it — and a policy of rescoring on majority agreement turns it into apparent **correctness**.

## Why it matters

Any grounding check *passes* it — because it is true. The entity is real, it was traversed, the triple exists. Verification cannot catch it by construction.

**The contribution is the mechanism, named, not the count.**

# Contributions

1. AGR and its **Structural Verification Layer**
2. A **component-level ablation** of four mechanisms
3. **Stratum-dependent decomposition** — planning hurts
4. The *echo attractor*, named
5. **Benchmark-defect rates**: 57 questions whose gold labels are wrong or ambiguous
6. **Pre-specified** thresholds, fixed in advance

## What I would not claim

- The verifier logs only what it **re-jects**: wrongful acceptance is unmeasured, and the evidence is not persisted
- One environment, one backbone
- ToG leads where it finishes, from a **narrower candidate set**: 40/20 vs 300/200

**Thank you**

Questions